



INFORMATION COMMUNICATION AND TECHNOLOGY UNIVERSITY (ICT-U)

Yaoundé Cameroon

WHICHEMAIL – AI-POWERED EMAIL CREDENTIAL MANAGER

FINAL ASSESSMENT PROJECT REPORT (ICT)

By:

FANYI CHARLLSON FANYI

(ICTU20233841)

CS 3410 INTRODUCTION TO MOBILE APPLICATION DEVELOPMENT

Lecturer:

Dr. Fotsing Kuetché

1. APP DESCRIPTION

Problem Statement

In today's digital landscape, users maintain multiple email addresses for different purposes—personal, work, school, and temporary accounts. A common frustration emerges when attempting to log into a service: "Which email did I use to sign up for this?" This seemingly simple question often leads to failed login attempts, unnecessary password resets, and security concerns from repeatedly using "forgot password" workflows.

Current solutions like password managers focus primarily on password storage but lack dedicated email tracking features. Users resort to mental recall, spreadsheets, or trial-and-error approaches, all of which are inefficient and error-prone. The problem becomes more acute as the average user manages accounts across 100+ different services.

Target Users

WhichEmail is designed for three primary user groups:

1. **Multi-email professionals** who juggle personal, work, and side-project email addresses
2. **Privacy-conscious users** who use different emails for different service categories (shopping vs. banking vs. social media)
3. **General smartphone users** who simply want to remember which email they used without the cognitive overhead

The app targets users aged 18-45 who are digitally active and manage accounts across numerous platforms. Based on informal surveys during development, I found that 8 out of 10 peers had experienced the "which email?" problem within the past week.

Core Features

WhichEmail provides a secure, encrypted vault with the following capabilities:

Service Management: Users can create, read, update, and delete service entries with associated emails. Each service includes optional fields for website URLs, passwords, and custom notes.

Smart Organization: Services are categorized (Social Media, Banking, Shopping, Entertainment, etc.) with visual icons and color coding. Users can mark frequently accessed services as favorites for quick retrieval.

Secure Password Storage: Passwords are encrypted using AES-256 encryption and stored in the device's secure enclave. Biometric authentication (Face ID/fingerprint) is required to view or copy passwords, ensuring that even if the device is compromised, passwords remain protected.

AI Assistant: Integrated Google Gemini API provides contextual help—suggesting strong passwords, recommending which email to use for new services based on usage patterns, and offering security tips.

2. DESIGN PROCESS

Initial Conceptualization

The design process began with user story mapping. I identified the core user journey: "As a user, I need to quickly find which email I used for Netflix so I can log in." This simple story drove the entire UX design—everything had to be accessible within 2-3 taps.

I sketched low-fidelity wireframes on paper, focusing on three primary screens: the service list (home), service details, and add/edit forms. Early feedback from classmates revealed that a search bar needed to be prominently placed, leading to its position at the top of the home screen.

Architecture Explanation

WhichEmail follows a **clean architecture pattern** with clear separation of concerns:

Presentation Layer (UI): Built with React Native and NativeWind (Tailwind CSS for React Native), the UI layer consists of reusable components organized in a component library. Expo Router handles file-based navigation, eliminating the need for manual route configuration.

Business Logic Layer: Custom React hooks (`useServices`, `useBiometrics`, `useTheme`) encapsulate business logic and provide clean interfaces to UI components. React Query manages server state and caching, while Zustand handles client-side state (authentication, theme preferences).

Data Layer: Appwrite serves as the Backend-as-a-Service, providing authentication, real-time database, and cloud synchronization. Locally, AsyncStorage handles non-sensitive data, while Expo SecureStore manages encrypted password storage.

Data Flow:

1. User interacts with UI component
2. Component calls custom hook (e.g., `useServices`)
3. Hook triggers React Query mutation
4. Mutation sends request to Appwrite via SDK
5. Appwrite responds with updated data
6. React Query updates cache
7. UI automatically re-renders with new data

This architecture ensures that components remain lightweight and testable, while business logic is centralized and reusable.

3. IMPLEMENTATION HIGHLIGHTS

Technical Choice Justifications

Why Expo over React Native CLI? I chose Expo because it provides a managed workflow that abstracts away native build configurations. For a time-constrained academic project, Expo's pre-built modules (SecureStore, LocalAuthentication) saved weeks of native development. The tradeoff is less control over native code, but for this project's scope, the benefit outweighed the cost.

Why NativeWind over StyleSheet? NativeWind brings Tailwind's utility-first CSS to React Native. This choice drastically improved development speed—I could style components inline without context switching to separate style objects. The dark mode implementation was particularly elegant: `className="bg-white dark:bg-slate-900"` handles both themes declaratively.

Meeting Course Requirements

Authentication: Email/password authentication via Appwrite, with biometric re-authentication for sensitive operations. Sessions persist in SecureStore, and tokens refresh automatically when they approach expiration.

UI/UX: Designed with accessibility in mind—all interactive elements have descriptive `accessibilityLabel` props, color contrast meets WCAG AA standards, and dark mode is fully implemented. Animations use React Native Reanimated for 60fps smooth transitions.

4. TESTING

Testing Strategy

Testing was approached with a pragmatic mindset: focus on critical paths and business logic rather than achieving 100% coverage. I used Jest for unit and integration testing, with React Native Testing Library for component testing.

Unit Tests:

Utility functions: Validation logic (email format, password strength), encryption/decryption helpers, date formatting functions.

Business logic hooks: Tested `useServices` hook to ensure proper data transformations and error handling. Mocked

Manual Testing Process: Manual testing covered user workflows that are difficult to automate:

1. **Onboarding flow:** Verified welcome screen shows only on first launch, then never again.

2. **Navigation:** Tested all screen transitions, back button behavior, and deep linking (opening a service detail directly via notification, though notifications aren't implemented yet).

Debugging Process

During development, I encountered several bugs that required systematic debugging:

Bug #1: Biometric prompt not appearing

Issue: On Android, the biometric prompt would fail silently when attempting to view passwords.

Diagnosis: Used Android Studio Logcat to discover a permissions error—`USE_BIOMETRIC` wasn't declared in `AndroidManifest.xml`.

Solution: Added the permission via Expo's app.json config plugin. This taught me to always check platform-specific requirements.

5 REFLECTION

Challenges Faced

Challenge 1: Managing Asynchronous State The most persistent challenge was coordinating asynchronous operations—biometric authentication, network requests, and local storage all happen asynchronously, and orchestrating them without race conditions required careful thought. For example, when viewing a password, the app must: (1) trigger biometric prompt, (2) wait for success, (3) decrypt password from SecureStore, (4) display it. If the user navigates away mid-process, the operation should cancel gracefully.

How I overcame it: I adopted React Query's pattern of treating all async operations as "queries" or "mutations." This centralized error handling and loading states. I also used `useEffect` cleanup functions to cancel pending operations when components unmounts.

Conclusion

WhichEmail started as a solution to a personal frustration and evolved into a comprehensive mobile application demonstrating the concepts learned in CS 3410. The project challenged me to balance user needs, technical constraints, and time limitations—a microcosm of real-world software development.